

Problem 1: Declaring Input and Output in 8086 Processor

Objective: To understand and demonstrate the fundamental assembly language instructions and DOS interrupts (INT 21H) required to display text (output) to the screen and read a character (input) from the keyboard in an 8086 microprocessor environment.

Source Code:

```
.MODEL SMALL
```

```
.STACK 100H
```

```
.DATA
```

```
    MSG1 DB 'Enter a character: $'
```

```
    MSG2 DB 0DH,0AH,'You entered: $'
```

```
.CODE
```

```
MAIN PROC
```

```
    MOV AX, @DATA
```

```
    MOV DS, AX
```

```
    MOV DX, OFFSET MSG1
```

```
    MOV AH, 09H
```

```
    INT 21H
```

```
    MOV AH, 01H
```

```
    INT 21H
```

```
    MOV BL, AL
```

```
    MOV DX, OFFSET MSG2
```

```
    MOV AH, 09H
```

```
    INT 21H
```

```
    MOV DL, BL
```

```
    MOV AH, 02H
```

```
    INT 21H
```

```
    MOV AH, 4CH
```

```
INT 21H  
MAIN ENDP  
END MAIN
```

Output



Figure 1: Taking Input and Showing Output

Problem 2: Declaring Variables in 8086 Processor

Objective: To learn the syntax and directives used to declare variables (data) of different sizes in the .DATA segment of an 8086 assembly program. This includes using DB (Define Byte), DW (Define Word), and DD (Define Doubleword).

Source Code:

```
.MODEL SMALL  
  
.STACK 100H  
  
.DATA  
  
CR EQU 13  
LF EQU 10  
NUM DB ?  
  
.CODE  
  
STARTPROC PROC  
  
    MOV AX, @DATA  
  
    MOV DS, AX  
  
    MOV AH, 1  
  
    INT 21H  
  
    MOV NUM, AL  
  
    MOV DL, CR  
  
    MOV AH, 2
```

```

INT 21H

MOV DL, LF

INT 21H

MOV DL, NUM

MOV AH, 2

INT 21H

MOV AH, 4CH

INT 21H

STARTPROC ENDP

END STARTPROC

```

Output:

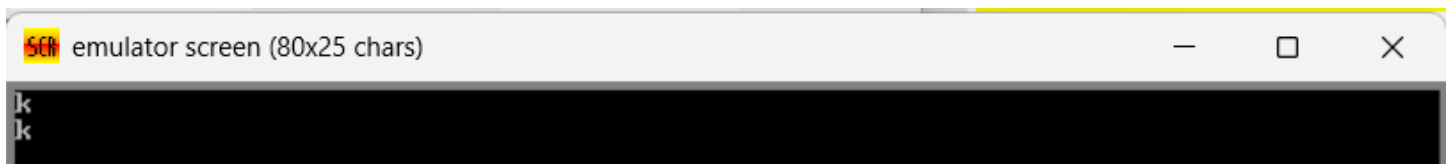


Figure 2: Declaring variables

Problem 3: Adding Two Numbers Without Variables in 8086 Processor

Objective: To perform arithmetic addition of two 8-bit numbers using only CPU registers and immediate values (hard-coded numbers), without defining any variables in the data segment.

Source Code:

```

.MODEL SMALL

.STACK 100H

.DATA

CR EQU 13

LF EQU 10

.CODE

STARTPROC PROC

    MOV AX, @DATA

    MOV DS, AX

```

```
MOV AH, 1
INT 21H
SUB AL, '0'
MOV BL, AL
MOV DL, CR
MOV AH, 2
INT 21H
MOV DL, LF
INT 21H
MOV AH, 1
INT 21H
SUB AL, '0'
MOV BH, AL
MOV DL, CR
MOV AH, 2
INT 21H
MOV DL, LF
INT 21H
MOV AL, BL
ADD AL, BH
ADD AL, '0'
MOV DL, AL
MOV AH, 2
INT 21H
MOV AH, 4CH
INT 21H
STARTPROC ENDP
```

END STARTPROC

Output

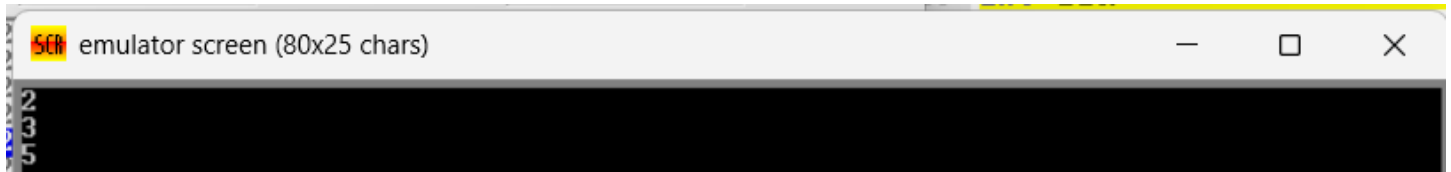


Figure 3: Adding two Numbers

Problem 4: Subtracting Two Numbers Without Variables in 8086 Processor

Objective: To perform arithmetic subtraction of two 8-bit numbers using only CPU registers and immediate values, without defining any variables in the data segment.

Source Code:

```
.MODEL SMALL  
  
.STACK 100H  
  
.DATA  
  
CR EQU 13  
LF EQU 10  
  
.CODE  
  
STARTPROC PROC  
  
    MOV AX, @DATA  
  
    MOV DS, AX  
  
    MOV AH, 1  
  
    INT 21H  
  
    SUB AL, '0'  
  
    MOV BL, AL  
  
    MOV DL, CR
```

```
MOV AH, 2
INT 21H
MOV DL, LF
INT 21H
MOV AH, 1
INT 21H
SUB AL, '0'
MOV BH, AL
MOV DL, CR
MOV AH, 2
INT 21H
MOV DL, LF
INT 21H
MOV AL, BL
SUB AL, BH
ADD AL, '0'
MOV DL, AL
MOV AH, 2
INT 21H
MOV AH, 4CH
INT 21H
STARTPROC ENDP
END STARTPROC
```

Output:

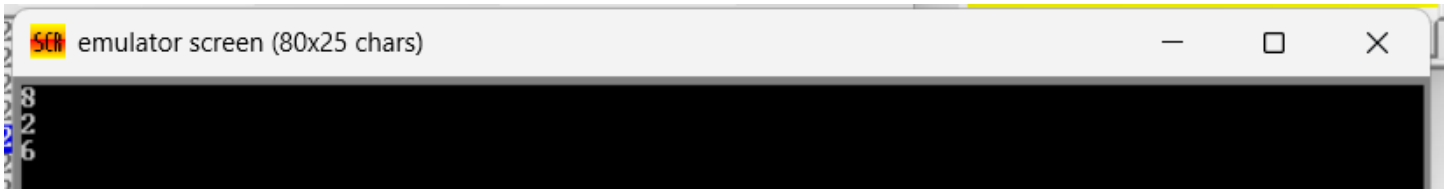


Figure 4: Subtracting two numbers

Conclusion

This lab provided hands-on experience with four fundamental concepts of 8086 assembly programming. We learned:

1. **Problem 1:** How to interact with the user by displaying messages (output) and reading keyboard strokes (input) using DOS INT 21H services.
2. **Problem 2:** The correct syntax for allocating memory for data, defining variables of different sizes (DB, DW, DD), and initializing them in the .DATA segment.
3. **Problem 3 & 4:** How to perform basic arithmetic operations (Addition and Subtraction) using CPU registers (AL, BL) and immediate values. This demonstrates the processor's ability to manipulate data directly without relying on memory variables.